Islamic University of Technology

Lab 01: Empirical Analysis & The Master Method

CSE 4810: Algorithm Engineering

2A

ASH, ABT(PT)

August 6, 2026

General Instructions: Empirical Analysis

For your assigned task, we expect you to engineer a solution that not only proves theoretical correctness but also demonstrates empirical performance constraints.

1. **Implementation:** Implement the two algorithms specified in your assigned set.
2. **Instrumentation:** Use high-resolution timers (e.g., `std::chrono` in C++, `System.nanoTime()` in Java, or `time.perf_counter()` in Python) to measure execution time.
3. **Data Recording:** Run both algorithms over an increasing series of input sizes (n). Save your results to a text file (`data.txt`) in three columns: `n`, `Time_Incremental`, `Time_DC`.
4. **Visualization:** Use `gnuplot` via the Ubuntu terminal to plot your data. Identify the exact **Crossover Point** where the divide-and-conquer approach theoretically overtakes the incremental approach.
5. **Theoretical Analysis:** Derive the asymptotic time complexity of the Divide and Conquer algorithm using the Master Method or Recursion Tree.

Set 2: Subarray Optimization

Problem Statement: You are given an array a consisting of n integers. Your task is to find the maximum possible sum of a non-empty contiguous subarray.
Formally, find the maximum value of:

$$\sum_{k=i}^{j} a_k$$

where $1 \leq i \leq j \leq n$.

Tasks:

- **Incremental Task:** Implement the **Naive Maximum Subarray** algorithm ($O(n^2)$) using two nested loops to evaluate the sums of all possible subarrays.
- **Divide & Conquer Task:** Implement the **D&C Maximum Subarray** algorithm ($O(n \log n)$).
- **Engineering Goal:** Plot execution time versus n for n up to at least $10,000$. Identify the crossover point. Observe how the $O(n^2)$ approach scales prohibitively compared to the $O(n \log n)$ model.

Supplemental Material: Maximum Subarray (D&C)

To achieve $O(n \log n)$, we must check the left half, the right half, and the subarray that crosses the midpoint.

Finding the Crossing Subarray:

```
Find-Max-Crossing-Subarray(A, low, mid, high)
    left_sum = -infinity, sum = 0
    FOR i = mid DOWN TO low:
        sum = sum + A[i]
        IF sum > left_sum: left_sum = sum

    right_sum = -infinity, sum = 0
    FOR j = mid + 1 TO high:
        sum = sum + A[j]
        IF sum > right_sum: right_sum = sum

    RETURN (left_sum + right_sum)
```

Student Manual: High-Resolution Timing and Python Pyplot

This manual provides the standard operating procedures for timing your code in C++, Java, and Python, followed by instructions for plotting your results using Python's `matplotlib.pyplot` library.

Part 1: Instrumentation (Time Calculation)

CRITICAL RULE: Never place I/O operations (like `printf`, `cout`, or `print`) inside your timing blocks. Console output is thousands of times slower than basic arithmetic and will completely corrupt your empirical data.

1. C++ (Using `std::chrono`)

Do not use the legacy C-style `clock()` function, as it can be inaccurate on modern multi-core systems. Use the C++11 `chrono` library for high-resolution wall-clock time.

```
#include <iostream>
#include <chrono>
```

```
using namespace std;
using namespace std::chrono;

int main() {
    int n = 1000;
    // ... initialize your array/data here ...

    // Start timer
    auto start = high_resolution_clock::now();

    // Execute ONLY the algorithm
    myAlgorithm(arr, n);

    // Stop timer
    auto stop = high_resolution_clock::now();

    // Calculate duration in microseconds
    auto duration = duration_cast<microseconds>(stop - start);

    cout << "Time taken: " << duration.count() << " microseconds" << endl;

    return 0;
}
```

2. Java (Using `System.nanoTime()`)

Avoid `System.currentTimeMillis()`, as its resolution is often tied to the operating system's time-slicing granularity.

```
public class Profiler {
    public static void main(String[] args) {
        int n = 1000;
        // ... initialize your array/data here ...

        // Start timer
        long startTime = System.nanoTime();

        // Execute ONLY the algorithm
        myAlgorithm(arr, n);

        // Stop timer
        long endTime = System.nanoTime();

        // Calculate duration in microseconds
        long duration = (endTime - startTime) / 1000;

        System.out.println("Time taken: " + duration + " microseconds");
    }
}
```

3. Python (Using `time.perf_counter()`)

Do not use `time.time()`. Use `time.perf_counter()` which provides the highest available resolution clock for short durations.

```
import time

n = 1000
# ... initialize your array/data here ...

# Start timer
start_time = time.perf_counter()

# Execute ONLY the algorithm
my_algorithm(arr, n)

# Stop timer
end_time = time.perf_counter()

# Calculate duration in microseconds
duration = (end_time - start_time) * 1_000_000

print(f"Time taken: {duration:.2f} microseconds")
```

Part 2: Data Formatting

Your program should run your algorithms inside a loop that gradually increases n (e.g., $n = 10, 50, 100, 500, 1000 \ldots$). Instead of printing to the console, write the results directly to a text file named `data.txt`. Ensure the data is formatted in three distinct columns separated by spaces:

1. Input size (n)
2. Time taken by the Incremental algorithm (in microseconds)
3. Time taken by the Divide and Conquer algorithm (in microseconds)

Example `data.txt`:

```
# n       Time_Incremental   Time_DC
10        2                  4
50        15                 18
100       65                 35
500       1600               180
1000      6450               385
```

Part 3: Plotting with Python Pyplot

Once you have generated your `data.txt` file, you will use Python and the `matplotlib` library to visualize your asymptotic curves and find the crossover point.

1. Prerequisites

If your Ubuntu lab PC does not already have `matplotlib` installed, open your terminal and install it using the system package manager:

```
sudo apt update
sudo apt install python3-matplotlib
```

2. The Plotter Script

Create a new file named `plotter.py` in the same directory as your `data.txt` file. Copy and paste the following Python code into it:

```
import matplotlib.pyplot as plt

n_vals, inc_times, dc_times = [], [], []

# Read data from the text file
with open('data.txt', 'r') as f:
    for line in f:
        # Ignore headers/comments and empty lines
        if line.startswith('#') or not line.strip():
            continue
        parts = line.split()
        n_vals.append(float(parts[0]))
        inc_times.append(float(parts[1]))
        dc_times.append(float(parts[2]))

# Create the plot
plt.figure(figsize=(10, 6))
plt.plot(n_vals, inc_times, label='Incremental Algorithm', marker='o', color='re
plt.plot(n_vals, dc_times, label='Divide & Conquer Algorithm', marker='x', color

# Formatting the graph
plt.title('Algorithm Engineering: Empirical Analysis Comparison')
plt.xlabel('Input Size (n)')
plt.ylabel('Execution Time (microseconds)')
plt.legend()
plt.grid(True)

# Display the interactive window
plt.show()
```

3. Running the Script & Finding the Crossover Point

Run the script from your terminal:

```
python3 plotter.py
```

When the interactive Pyplot window opens:

- You will likely see the Incremental curve ($O(n^2)$) shoot upward rapidly, squashing the Divide & Conquer curve against the bottom axis.

- To find the exact **crossover point** (where the two lines intersect), locate the navigation toolbar at the bottom of the window.
- Click the **Magnifying Glass icon** (Zoom to rectangle).
- Click and drag a box over the intersection of the two lines (usually very close to the bottom-left corner of the graph) to zoom in.
- You may need to zoom in a few times to identify the exact value of n where the Divide & Conquer algorithm officially becomes faster.
- Click the **Home icon** to reset the view to the original scale.